# Individual Project

# 📌 Graded Individual Project

**📅 Final Submission Deadline: 10th April**

---

## 🔰 Introduction

The objective of this project is to develop a web application that demonstrates your understanding of JavaScript, API integration, and UI development.

The application should include:

- Integration of a public API (using fetch)
- Use of array HOFs for filtering, sorting, and searching
- A well-designed user interface using CSS

💡 You may use libraries such as Tailwind CSS or Bootstrap if preferred (optional, not mandatory).

You are encouraged to explore creative ideas and enhance their projects with additional features and styling.

⚠️ Use of AI tools is strictly prohibited. Any violation will result in deduction of marks.

🔍 **Finding Public APIs**
You can explore a wide range of public APIs for your project using the following resources:

- https://github.com/public-apis/public-apis
- https://github.com/public-api-lists/public-api-lists

📅 **Final Deadline: 10th April**
📊 The project will be evaluated at each milestone.

# 🚀 Milestone 1: Project Setup and Basic Structure

## 🎯 Goal

You are required to decide what you will build and plan the foundation of your project.

## ✅ Tasks

- Choose a **project idea** for your web application
- Select a **public API** that you will use in your project
- Create a **GitHub repository** for your project
- Create a **README.md** file
    - The **README** file should clearly describe your project, including its purpose, the API being used, the features you plan to implement, and the technologies involved. It should also provide basic information on how the project can be set up and run.

## 💡 Important Notes

- This milestone focuses on **planning and clarity of your idea**, not implementation
- Choose a project that is **feasible and can be completed within the given timeline**
- Ensure your idea supports features like **search, filtering, or sorting** for later milestones

📅 **Deadline: 23rd March**

# 🌐 Milestone 2 - API Integration

## 🎯 Goal

To integrate and display real-time data from a public API.

## ✅ Tasks

- Implement API calls using fetch
- Display the fetched data dynamically on the webpage
- Handle loading states effectively
- Ensure the website is fully **responsive across devices** (mobile, tablet, desktop)

## 📦 Deliverables

- Successful integration and display of API data

📅 **Deadline: 1st April**

# ⚙️ Milestone 3 - Core Features

## 🎯 Goal

To enhance the application by adding interactivity and dynamic behavior.

## ✅ Tasks

Implement at least three of the following features:

- **Searching:**
  Enable users to search data based on relevant keywords
- **Filtering:**
  Allow users to filter results based on specific criteria
- **Sorting:**
  Provide options to arrange data (e.g., ascending, descending, alphabetical)
- **Button Interactions:**
  Include interactive elements such as like, favorite, or view more
- **Dark Mode / Light Mode:**
  Implement a theme toggle for better user experience

## ❗ Important Requirement

- Searching, filtering, and sorting must be implemented using **Array Higher-Order Functions** (such as `map`, `filter`, `reduce`, `find`, `sort`, etc.)
- Do **not** use traditional loops like `for` or `while` for these operations

## 📦 Deliverables

- Fully functional interactive features

📅 **Deadline: 8th April**

# 🚀 Milestone 4 - Documentation, Deployment, and Final Submission

## 🎯 Goal

To finalize, document, and deploy the project.

## ✅ Tasks

- Update the **README.md** with final details
- Refactor and clean the codebase
- Deploy the project using a suitable platform.

📅 **Final Submission Deadline: 10th April**

# ⭐ Bonus Features (Optional but Recommended)

You can enhance your project by implementing the following features. These are optional but will improve performance, usability, and overall user experience.

- **Debouncing**
  Limits how frequently a function is executed, especially useful for search inputs.
  👉 Can be applied to the **search bar** to avoid making API calls on every keystroke.

- **Throttling**
  Ensures a function runs at a controlled rate over time.
  👉 Can be used during **scrolling events** or repeated button clicks to improve performance.

- **Pagination**
  Divides large sets of data into smaller, manageable pages.
  👉 Useful when displaying **lists of results from APIs** (e.g., products, movies, users).

- **Loading Indicators**
  Provides visual feedback while data is being fetched.
  👉 Can be shown during **API calls** to indicate that content is loading.

- **Local Storage**
  Allows storing data in the browser for persistence.
  👉 Can be used to save **favorites, liked items, or user preferences** (e.g., dark mode).

- **Infinite Scroll**
  Automatically loads more data as the user scrolls down.
  👉 Can replace pagination for **continuous content loading** in lists or feeds.

- **Progressive Web App (PWA)**
  Enhances your web application to behave like a native app with offline support and installability.
  👉 Can be implemented by adding a service worker and manifest file to enable features like offline access, caching API data, and allowing users to install the app on their device.

💡 **Note:**
Implementing one or more of these features will strengthen your project and demonstrate advanced JavaScript concepts.

# ✅ Best Practices

To ensure a smooth development process and better project quality, follow these best practices:

- **Make Regular Commits**
  Commit your code frequently with meaningful messages (e.g., "`added search functionality`", "`implemented API fetch`").

- **Follow Clean Code Practices**
  Write readable and maintainable code with proper indentation and formatting.

- **Use Meaningful Names**
  Use clear and descriptive variable and function names.

- **Avoid Code Repetition**
  Reuse functions and components wherever possible (DRY principle).

- **Maintain Proper Structure**
  Organize your code by separating concerns (API logic, UI rendering, event handling).

- **Handle Errors Gracefully**
  Add basic error handling for API calls (e.g., network issues, empty data).

- **Ensure Responsiveness**
  Continuously check your UI on different screen sizes during development.

- **Keep Your README Updated**
  Update documentation as you progress, not just at the end.

- **Write Modular Code**
  Break your code into smaller, reusable functions for better readability and maintainability.

# 💪 Final Note

This project is a great opportunity to apply everything you've learned and build something you can be proud of. Focus on understanding concepts, writing clean code, and improving step by step with each milestone.

Don't hesitate to experiment, make mistakes, and learn from them — that's an important part of the development process.

🚀 **All the best!** We're excited to see the ideas you come up with and the projects you build. Give it your best effort and make it count!

# Sample Project Ideas

1. <u>NASA Mission Explorer</u>

   Description: A high-end gallery that fetches NASA's "Astronomy Picture of the Day" (APOD). You can create a feature where users can pick a date to see what the universe looked like on their birthday.

   Core features: Date picker for historical data, full-screen image modal and "HD" toggle.

   UI Ideas: Dark "**Space**" theme with glowing borders and high-resolution background imagery.

   ***Public API:*** https://api.nasa.gov/

2. <u>Retro Game Vault</u>

   Description: A search engine for classic and modern video games. Can build a "Wishlist" feature that saves games to the browser's memory.

   Core Features: Search bar with "**debounce**" logic, platform filtering (PC, PlayStation, etc.) and a "**Favorites**" sidebar.

   ***Public API:*** https://rawg.io/apidocs

   UI Ideas: Grid layout with large game box art and star ratings.

   Challenge: Add a "Compare" mode to see specs of two games side-by-side.

3. <u>Anime Next Watch Generator</u>

   Description: For people who can't decide what to watch. This app fetches top-trending anime and lets users filter by genre or "Top Rated."

   Core Features: Trending list, genre-based search, and a "Synopsis" expandable card.

   ***Public API:*** https://docs.api.jikan.moe

4. <u>Smart City Weather Dashboard</u>

Description: Beyond just temperature, this app provides "What to Wear" advice based on weather conditions (e.g., "Bring an umbrella!" if rain > 20%).

Core Features: Search by city, 5-day forecast, and dynamic icons based on weather codes.

***Public API:*** https://openweathermap.org/api

UI Ideas: Background colors that change based on temperature (blue for cold, orange for hot).

Challenge: Use the browser's Geolocation API to load the local weather automatically

5. Global Currency "Jetsetter"
   Description: A travel-focused currency converter. Users can select multiple "Target Currencies" to see how their money stretches across different countries simultaneously.
   Core Features: Multi-select currency conversion, "Last Updated" timestamp, and a base-currency toggle.

   ***Public API:*** https://frankfurter.dev/

   UI Ideas: Clean, financial-app aesthetic with flag icons for each currency.

   Challenge: Display a historical sparkline (simple CSS bar chart) of the rate over the last 7 days.

6. The Meme Factory
   Description: A meme browser where users can search for famous templates and see how they are used.

   Core Features: Searchable meme grid, "Share" button (copy link to clipboard), and category tabs, "Back to Top" button logic, and handling image aspect ratios.

   ***Public API***: https://imgflip.com/api

   UI Ideas: A Masonry grid layout (like Pinterest) to handle different image sizes.

Challenge: Implement a "Meme Creator" preview where users type text into inputs and it appears over the image.

7. Tech News "Pulse"
   Description: A simplified news aggregator focusing on technology and startups. Students will learn to handle "Read More" links and image fallbacks.

   Core Features: Category filters (AI, Crypto, Gadgets), search functionality, and a "Save for Later" list.

   Public API: https://newsapi.org/

   Challenge: Add a "Night Mode" toggle that persists in localStorage.

8. Open Library Book Finder
   Description: A digital library where users can search for any book ever written. Great for practicing complex data structures.

   Core Features: Search by title/author, cover image display, and "Publish Year" sorting.

   Public API: https://openlibrary.org/developers/api

   React Concepts: Handling empty search states, sorting arrays of objects, and debouncing.

   UI Ideas: "Bookshelf" grid view with a flip animation to see the description on the back.

   Challenge: Create a "Reading Goal" counter that increments when you click "I've read this."

9. Breed-o-Pedia (Dogs)
   Description: A deep dive into every dog breed. Users select a breed to see a gallery of images and sub-breed information.

   Core Features: Nested dropdowns (Breed -> Sub-breed), "Load More" images, and a random image generator.

   ***Public API:*** https://dog.ceo/dog-api/

   React Concepts: Dependent dropdowns (updating one based on another), and managing loading states for images.

UI Ideas: Large, friendly typography and a "Card" for every dog image.

Challenge: Add a "Vote" system (Hot or Not) for dog photos.

10. <u>Meal Master (Recipe Finder)</u>
    Description: A recipe search tool where users can input what's in their fridge to find meals they can actually make.

    Core Features: Filter by ingredient, view cooking instructions, and "Nutrition" facts toggle.

    ***Public API:*** https://www.themealdb.com/api.php

    React Concepts: URL parameters for "View Recipe" pages, and mapping ingredients list.

    UI Ideas: A clean, kitchen-inspired UI with high-quality food photography.

    Challenge: Add a "Shopping List" feature that compiles ingredients from 3 different recipes.

11. <u>The Daily Zen (Inspiration)</u>
    Description: A mental health dashboard that provides a beautiful quote, a serene background image, and a "Focus Task" for the day.

    Core Features: Random quote generator, Unsplash background integration, and a simple one-item todo.

    ***Public API:*** https://unsplash.com/developers
    and https://docs.zenquotes.io/zenquotes-documentation/

    React Concepts: Combining two different API responses into a single UI view, and useEffect with empty dependency arrays.

    UI Ideas: Ultra-minimalist, "Zen" interface with fading transitions.

    Challenge: Add a 1-minute "Breathing Circle" animation that expands and contracts.

12. <u>The Movie Watchlist Project</u>

*OMDB API*: https://www.omdbapi.com/

*TMDB API:* https://developer.themoviedb.org/docs/getting-started

*Watchmode API*: https://api.watchmode.com/

Core Features Students Must Build:

- Live Search: Fetch results from the OMDb API as the user types (or on click).
- Watchlist Toggle: A button on each movie card that adds/removes it from the list.

Empty States: Clear UI for when no movies are found or the watchlist is empty.

UI Ideas: A dark, sleek theme like Disney+ or HBO Max. Use a sticky sidebar for the "Saved Movies" so the user can see their list grow while they search.

Optional Challenge: Add a "Random Movie Night" button that picks one movie from their saved watchlist at random to watch.

## LIST OF Public API'S to choose from

https://github.com/public-apis/public-apis